CSA0609 – DESIGN AND ANALYSIS OF ALGORITHMS

CO1 – ASSESSMENT TOOL 1

Name: Hemanth P

Reg. No.: 192524297

Assignment Overview

This assignment analyses the scalability of ShopSphere, an online shopping platform. The catalogue contains 100,000 products today and is expected to grow to 1,000,000 products. The audit examines the number of basic operations performed by five back-end modules, solves the given recurrences, compares search cases, proves an asymptotic bound, and makes a practical redesign decision.

Assessment requirements

- Identify the input size and dominant basic operation for each module.
- Derive the exact comparison count for the duplicate-listing detector and exact audit count for the rate limiter.
- Solve the binary-search recurrence using backward substitution.
- Apply the Master Theorem to both recommendation-engine designs and analyse the sales-analytics recurrence.
- Compare best, average, and worst cases for sequential and binary search.
- Prove that the report generator has $\Theta(n^2)$ cost and is not $O(n)$.
- Estimate operations and running times at $n = 1{,}000{,}000$ using 10 basic operations per second.

Assumptions used

The input size n is the number of catalogue products or records being processed. A comparison or Audit(Request[j]) is counted as one basic operation when the question defines it that way. For time estimates, the supplied processing rate of 10 basic operations per second is used. Logarithms are base 2 unless stated otherwise.

For recurrences, a constant-size base case is assumed when a numerical estimate is needed. Asymptotic results do not depend on the exact constant base-case cost. The numerical estimates are idealised operation-count estimates, not benchmarks of real hardware.

Problem Context and Input Growth

ShopSphere must process a catalogue that grows from n = 100,000 to n = 1,000,000. This is a tenfold increase in input size. An algorithm that performs linear work will generally perform about ten times as much work. A quadratic algorithm performs roughly one hundred times as much work. An n log n algorithm grows somewhat faster than linear, while a logarithmic algorithm grows very slowly.

Why the basic operation matters

A basic operation is the repeated operation that most directly determines running time. Examples in this problem include comparing two product titles and seller IDs, auditing one request, or comparing a search key with a catalogue entry. Counting these operations gives a machine-independent way to compare algorithms before implementation.

Asymptotic notation

- $O(g(n))$ gives an asymptotic upper bound.
- $\Omega(g(n))$ gives an asymptotic lower bound.
- $\Theta(g(n))$ gives a tight bound when both upper and lower bounds have the same growth rate.

For example, $\Theta(n^2)$ means that for sufficiently large n, the operation count is bounded above and below by positive constant multiples of n^2. It does not state an exact number of seconds; it describes how the work grows as the input becomes larger.

Tenfold growth illustration

If n increases from 100,000 to 1,000,000, a $\Theta(n^2)$ module may need about 100 times the work, whereas a $\Theta(n \log n)$ module needs approximately ten times the input growth multiplied by a modest logarithmic factor. This difference is why the audit must consider both mathematical complexity and the nightly deadline.

(a) Understand the Problem — Module M1

M1: Duplicate-Listing Detector

Input size: n products. The routine compares every product with every later product. The dominant basic operation is the pairwise check of Product[i].title and Product[j].title and, when titles match, the seller identifiers. For counting purposes, each pair checked by the IF condition is one comparison operation.

Where the operation occurs

The comparison occurs in the inner loop. For i = 0, j visits 1 through n − 1, so there are n − 1 comparisons. For i = 1, there are n − 2 comparisons. This continues until i = n − 2, where one comparison is made.

Growth expectation

The total is the sum (n − 1) + (n − 2) + … + 2 + 1. This is a triangular number, equal to n(n − 1)/2. The dominant term is $n^2/2$, so M1 has $\Theta(n^2)$ running time.

The algorithm only uses loop variables and a fixed number of temporary values, so its auxiliary working space is Θ(1), excluding the catalogue and any output list of marked duplicates. If marked duplicate IDs are stored separately, that output can require up to O(n) space.

Interpretation

Every additional product is compared with many existing products. A tenfold increase in catalogue size creates roughly one hundred times as many pair checks. M1 is therefore a high-priority redesign candidate.

(a) Understand the Problem — Modules M2 and M3

M2: API Rate Limiter

Input size: n requests in the largest audit window. The basic operation is Audit(Request[j]), which is executed once per inner-loop iteration. The outer variable i starts at 1 and doubles after each pass. The inner loop performs i audits on that pass.

The number of audits is 1 + 2 + 4 + 8 + … while i remains below n. These are geometrically increasing terms. Their sum is less than 2n, so the operation count is $\Theta(n)$, not $\Theta(n \log n)$. The fact that the outer loop runs logarithmically many times does not mean each pass performs n operations; the inner-loop lengths are different on each pass.

M3: Product Search

S1 scans an unsorted catalogue from the beginning. Its basic operation is comparing the requested key with a product entry. It may stop early if the product is found, but it may inspect all n entries if the product is last or absent. Its worst-case time is $\Theta(n)$.

S2 uses a sorted index and checks the middle entry. Each comparison discards approximately half of the remaining search interval. Its recurrence is $T(n) = T(n/2) + 1$ with $T(1) = 1$. The number of comparisons grows logarithmically, so S2 has $\Theta(\log n)$ time.

Why sorting matters

Binary search is only valid when the searched index is sorted according to the same key being searched. Maintaining the sorted index has a cost, but once it is available, individual lookups require far fewer comparisons than sequential scanning for a large catalogue.

(a) Understand the Problem — Modules M4 and M5

M4: Recommendation Engine

Current design: $T(n) = 4T(n/2) + n$. The dominant recursive work is represented by four subproblems, each of size n/2, plus n units of merging work. Replicated partitions cause four recursive calls. The recurrence will be quadratic.

Proposed design: $T(n) = 2T(n/2) + n$. Removing replication leaves two subproblems of half size, plus linear merging work. This recurrence is $\Theta(n \log n)$.

M5: Sales Analytics Engine

The recurrence is $T(n) = 2T(n/2) + n \log n$. The algorithm splits the data into halves and performs n log n work across a level as specified in the recurrence. The recursion contributes another logarithmic factor, resulting in $\Theta(n \log^2 n)$.

Nightly report generator

The separate profiling expression is $f(n) = 3n^2 + 10n \log n + 50$. The dominant term is $3n^2$ because n log n and a constant grow more slowly than n^2. Therefore $f(n) = \Theta(n^2)$. A formal proof is given in part (f).

Summary of expected growth

- M1: $\Theta(n^2)$ — pairwise comparisons.
- M2: $\Theta(n)$ — geometric audit passes.
- M3 S1: $\Theta(n)$ worst case; S2: $\Theta(\log n)$.
- M4 current: $\Theta(n^2)$; redesigned: $\Theta(n \log n)$.
- M5: $\Theta(n \log^2 n)$; report generator: $\Theta(n^2)$.

(b) M1 — Exact Comparison Count

The outer loop uses $i = 0, 1, \ldots, n - 2$. For a fixed i, the inner loop begins at $j = i + 1$ and ends at $n - 1$. The number of inner-loop iterations for that i is:

1. $n - 1 - (i + 1) + 1 = n - i - 1$
2. **C(n)** $= (n - 1) + (n - 2) + \ldots + 2 + 1$
3. **C(n)** $= (n - 1)n / 2$
4. **C(n)** $= n(n - 1)/2 = (n^2 - n)/2$

Step 1: Write the iteration counts

When $i = 0$, there are $n - 1$ comparisons. When $i = 1$, there are $n - 2$ comparisons. When $i = 2$, there are $n - 3$ comparisons. The final outer-loop value $i = n - 2$ gives one comparison.

Step 2: Use the arithmetic-series formula

The sum of the first m positive integers is $m(m + 1)/2$. Here $m = n - 1$, so:

Step 3: State the exact result and complexity

The highest-order term is $n^2/2$. Constant factors and the lower-order term $-n/2$ do not change the asymptotic class. Therefore the time complexity is $\Theta(n^2)$.

Space complexity

Only i, j, and a constant number of temporary values are needed, so auxiliary space is $\Theta(1)$. The product catalogue itself is input storage and is not counted as auxiliary space. If the program maintains a separate list of duplicate records, that output list may use $O(n)$ additional space.

(b) M1 — Worked Example and Meaning

Example with n = 5 products

For i = 0, compare product 0 with products 1, 2, 3, and 4: 4 comparisons. For i = 1, compare product 1 with products 2, 3, and 4: 3 comparisons. For i = 2, there are 2 comparisons. For i = 3, there is 1 comparison.

C(5) = 4 + 3 + 2 + 1 = 10

The formula confirms this: C(5) = 5(5 − 1)/2 = 5 × 4/2 = 10 comparisons.

Example with n = 100,000

C(100,000) = 100,000 × 99,999 / 2

= 9,999,900,000 / 2

= 4,999,950,000

=4,999,950,000 comparisons

Even at 100,000 products, the routine performs nearly five billion pair checks. Increasing the catalogue to one million products increases the count to nearly 500 billion. This is not a small constant-factor slowdown; it is a consequence of quadratic growth.

Possible redesign

A hash-based grouping strategy can group products by a combined key such as normalized title and seller ID. Expected processing can be Θ(n) with a suitable hash table, assuming expected constant-time operations and a carefully defined title-normalization policy. Sorting by the combined key is another option, typically Θ(n log n). Either approach avoids comparing every possible pair.

(b) M2 — Exact Audit Count

The outer loop takes the values 1, 2, 4, 8, …, 2^k while each value is less than n. Let k be the smallest integer such that 2^k ≥ n. The executed values are 2^0 through 2^(k−1).

Step 1: Count audits in each pass

At i = 1, the inner loop audits 1 request. At i = 2, it audits 2 requests. At i = 4, it audits 4 requests. The same pattern continues, so the total is a geometric sum.

= A(n) = 1 + 2 + 4 + … + 2^(k−1)

Step 2: Apply the geometric-series formula

= 1 + 2 + 4 + … + 2^(k−1) = 2^k − 1

Since k = ceil($\log_2$ n), the exact count for positive integer n is:

= A(n) = 2^ceil($\log_2$ n) − 1

Step 3: Decide whether Θ(n log n) is correct

The smallest power of two that is at least n is less than 2n for n > 1. Therefore 2^ceil($\log_2$ n) − 1 is less than 2n. Also, the final executed pass is a power of two that is at least n/2, so the total is at least proportional to n. Thus A(n) = Θ(n).

The team member's Θ(n log n) claim is incorrect. The inner loop does not run n times on every outer-loop iteration; it runs 1, 2, 4, 8, and so on. Summing these lengths gives a geometric series, not n repeated log n times.

(b) M2 — Verification for n = 1,024

Step 1: Identify the executed values

Since 1,024 = 2^{10}, the loop condition i < 1,024 permits i = 1, 2, 4, 8, 16, 32, 64, 128, 256, and 512. After the pass at 512, i doubles to 1,024 and the loop stops.

Step 2: Add the audit operations

A(1,024) = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512

= 1,023

Step 3: Verify using the formula

A(n) = 2^ceil($\log_2$ n) − 1

A(1,024) = 2^10 − 1 = 1,024 − 1 = 1,023

Both calculations produce 1,023 audit operations. This agrees with the geometric-series result and supports the Θ(n) classification.

Important distinction

The number of outer-loop passes is $\log_2$ n, but the work in each pass is not n. The total work is approximately n. A loop with log n passes and n operations on every pass would be Θ(n log n), but that is not the routine given in the question.

(c) S2 — Backward Substitution

The recurrence for the sorted-index search is T(n) = T(n/2) + 1, with T(1) = 1. The term T(n/2) represents searching the half that remains after the middle comparison. The +1 represents the current comparison.

Step 1: Expand once

- **T(n) = T(n/2) + 1**

Step 2: Expand the smaller recurrence

- **T(n/2) = T(n/4) + 1**

Substitute this into the first line:

- **T(n) = T(n/4) + 1 + 1 = T(n/4) + 2**

Step 3: Expand again

- **T(n/4) = T(n/8) + 1**
- **T(n) = T(n/8) + 3**

Each expansion halves the problem size and adds one comparison. After r expansions, the pattern is:

- **T(n) = T(n/2^r) + r**

Step 4: Reach the base case

Assume n is a power of two, so n = 2^k. The base case is reached when n/2^r = 1. Solving gives 2^r = n and r = $\log_2 n$.

- **T(n) = T(1) + $\log_2$ n**
- **T(n) = 1 + $\log_2$ n**

This is the exact closed form under the stated recurrence for powers of two. The dominant term is $\log_2 n$, so T(n) = Θ(log n).

(c) S2 — Complexity and Implementation Space

Time complexity

Each comparison removes about half of the remaining entries. The number of halvings needed to reduce n entries to one is $\log_2 n$. Therefore the comparison count grows logarithmically. The recurrence gives $T(n) = 1 + \log_2 n$ for powers of two, and the asymptotic result is $\Theta(\log n)$.

Iterative binary search

An iterative implementation updates low and high indices in a loop. It does not create a new function call for each halving step.

- Time complexity: $\Theta(\log n)$.
- Auxiliary space complexity: $\Theta(1)$, because only a fixed number of indices and a middle index are stored.

Recursive binary search

A recursive implementation calls itself on the remaining half of the index. Each call remains on the call stack until the base case is reached.

- Time complexity: $\Theta(\log n)$.
- Auxiliary space complexity: $\Theta(\log n)$, because the recursion stack contains one frame per halving level.

Practical interpretation

For a catalogue of one million products, $\log_2(1{,}000{,}000)$ is approximately 19.93. A binary search therefore needs around 20 comparisons in the worst case, subject to the precise array size and implementation. This is a dramatic improvement over up to one million comparisons for a sequential search.

The index must remain sorted. If catalogue updates make maintaining the index expensive, that update cost should be measured separately; it does not change the logarithmic comparison count for an individual binary search.

(d) M4 — Master Theorem for the Current Design

The current recommendation engine follows T(n) = 4T(n/2) + n. Compare it with the Master Theorem form T(n) = aT(n/b) + f(n).

Step 1: Identify the parameters

- a = 4 recursive subproblems.
- b = 2, because each subproblem has size n/2.
- f(n) = n, the linear merge work.

Step 2: Calculate the critical exponent

- **$n^{(\log_b a)} = n^{(\log_2 4)} = n^2$**

The non-recursive work f(n) = n is polynomially smaller than n^2. Specifically, $n = O(n^{(2-1)})$, so $f(n) = O(n^{(\log_b a - \varepsilon)})$ with $\varepsilon = 1$.

Step 3: Select the Master Theorem case

This satisfies Master Theorem Case 1: the non-recursive work is smaller than the critical term by a polynomial factor. Therefore:

- **$T(n) = \Theta(n^2)$**

Interpretation

The four recursive calls create substantial repeated work. Even though merging at a node is linear, the total number of recursive subproblems grows quickly. The replicated design therefore has quadratic running time.

(d) M4 — Master Theorem for the Redesign

The proposed design follows T(n) = 2T(n/2) + n. Removing replication reduces the number of recursive calls from four to two.

Step 1: Identify the parameters

- a = 2 recursive subproblems.
- b = 2, each of size n/2.
- f(n) = n, the linear merge work.

Step 2: Calculate the critical term

- **n^(log_b a) = n^($\log_2$ 2) = n**

Here f(n) = n matches the critical term n^($\log_2$ a). The two terms have the same order.

Step 3: Apply Master Theorem Case 2

For f(n) = Θ(n^(log_b a) log^k n) with k = 0, Case 2 gives an additional logarithmic factor:

- **T(n) = Θ(n log n)**

Effect when data doubles

For the current quadratic design, doubling n multiplies the dominant work by approximately $2^2 = 4$. For the redesigned n log n algorithm, the ratio is approximately $2n \log_2(2n)/(n \log_2 n) = 2(\log_2 n + 1)/\log_2 n$. This is slightly above 2 and approaches 2 as n becomes large.

The redesign is substantially more scalable because it removes replicated recursive work. It changes the growth class from quadratic to n log n.

(d) M5 — Extended Master Theorem

The sales analytics recurrence is T(n) = 2T(n/2) + n log n. The standard Master Theorem compares f(n) with n^(log_b a).

Step 1: Identify the parameters

- a = 2.
- b = 2.
- f(n) = n log n.

$n^{(\log_b a)} = n^{(\log_2 2)} = n$

The non-recursive term is n log n, which is larger than n by a logarithmic factor. It does not fit the simplest standard Case 1 or the k = 0 version of Case 2. Use the extended Case 2 form.

Step 2: Match the extended form

The extended Master Theorem handles f(n) = Θ(n^(log_b a) log^k n). Here n^(log_b a) = n and k = 1 because f(n) = n log n.

Step 3: Apply the result

- **$T(n) = \Theta(n^{(\log_b a)} \log^{(k+1)} n)$**
- **$T(n) = \Theta(n \log^2 n)$**

Recursion-tree explanation

At a level containing subproblems of total size n, the work associated with the n log n term is proportional to n log n at the root. At the next level, the total subproblem size remains n, while each subproblem has about half the original logarithmic size, giving a level cost proportional to n(log n − 1). The costs decrease by one n-sized unit per level.

There are Θ(log n) levels. Adding the level costs gives n[log n + (log n − 1) + … + 1], which is $\Theta(n \log^2 n)$. This independently confirms the extended Master Theorem result.

(e) Best, Average and Worst Cases — S1

S1 is sequential search through an unsorted catalogue. It checks entries from the first position onwards. The number of comparisons depends on where the product appears or whether it appears at all.

Best case

The searched product is at the first position. One comparison identifies it.

- $C_{best}(n) = 1 = \Theta(1)$

Worst case

The product is at the last position, or it is absent. The search examines all n entries.

- $C_{worst}(n) = n = \Theta(n)$

Average successful search

Assume the target is equally likely to occur at any of the n positions. The comparison counts are 1, 2, 3, …, n. Their average is:

- $C_{success_avg}(n) = (1 + 2 + \ldots + n)/n$
- $= [n(n + 1)/2]/n = (n + 1)/2$

This is $\Theta(n)$. If the product is guaranteed to exist and its position is uniformly distributed, the average number of comparisons is $(n + 1)/2$.

Unsuccessful search

If the product does not exist, S1 checks all n entries before concluding that it is absent. The comparison count is n.

(e) S1 — Average Cost with Existence Probability p

Let p be the probability that the searched product exists. When it exists, assume each of the n positions is equally likely, so the conditional average is (n + 1)/2. When it does not exist, all n entries are checked.

Step 1: Write the weighted average

- $E[C] = p \times (n + 1)/2 + (1 - p) \times n$

Step 2: Expand the terms

- $E[C] = p(n + 1)/2 + n - pn$

Step 3: Simplify

- $E[C] = n - p(n - 1)/2$

This formula combines successful and unsuccessful searches according to their probabilities. If p = 1, it becomes (n + 1)/2. If p = 0, it becomes n, which is the unsuccessful-search cost.

Evaluate for n = 1,000,000 and p = 0.5

E[C] = 1,000,000 − 0.5(1,000,000 − 1)/2

= 1,000,000 − 0.5 × 999,999 / 2

= 1,000,000 − 249,999.75

= 750,000.25 comparisons (expected value)

The fractional result is an expected value across many searches, not a possible count for one individual search. The result is about 750,000 comparisons per query, showing that sequential search is costly for a large catalogue.

(e) Best, Average and Worst Cases — S2

S2 is binary search on a sorted index. Each comparison checks the middle entry and either succeeds or discards about half the remaining entries.

Best case

The requested product is exactly at the middle position of the current search interval on the first comparison.

- **$C_{best}(n) = 1 = \Theta(1)$**

Worst case

The target is found only after the interval has repeatedly been halved, or the search reaches an empty interval and reports that the key is absent. The number of comparisons is logarithmic.

- **$C_{worst}(n) = \Theta(\log_2 n)$**

A common upper bound for a standard binary search over n sorted entries is approximately $\text{ceil}(\log_2(n + 1))$ comparisons, with details depending on the exact loop and termination convention. For n = 1,000,000, this is at most about 20 comparisons.

Average case

For targets distributed across the sorted index, the average comparison count is also $\Theta(\log n)$. Exact averages depend on the distribution of target keys and the shape of the binary-search decision tree. The important asymptotic result is logarithmic growth.

Comparison of S1 and S2

S1 is easy to use on an unsorted list but can take n comparisons. S2 needs sorted data, but it usually needs around 20 comparisons for one million entries. Therefore S2 is the preferred search method when ShopSphere can maintain a suitable sorted index.

(f) Prove $f(n) = \Theta(n^2)$

The report generator cost is $f(n) = 3n^2 + 10n \log_2 n + 50$. To prove $\Theta(n^2)$, use the formal definition: find positive constants c_1, c_2 and n_0 such that for every $n \geq n_0$, $c_1 n^2 \leq f(n) \leq c_2 n^2$.

Step 1: Establish the lower bound

For $n \geq 1$, both $10n \log_2 n$ and 50 are non-negative. Therefore:

- **$f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$**

Choose $c_1 = 3$. Then $c_1 n^2 \leq f(n)$ for every $n \geq 1$.

Step 2: Establish the upper bound

For $n \geq 64$, $\log_2 n \leq n/10$. This inequality holds at $n = 64$ because $6.0 \leq 6.4$, and the right side grows faster thereafter. Also, $50 \leq n^2$ for $n \geq 8$. Therefore, for $n \geq 64$:

- **$10n \log_2 n \leq n^2$**
- **$50 \leq n^2$**

Substitute these inequalities into $f(n)$:

- **$f(n) = 3n^2 + 10n \log_2 n + 50$**
- **$\leq 3n^2 + n^2 + n^2 = 5n^2$**

Choose $c_2 = 5$. The upper bound is satisfied for all $n \geq 64$.

Step 3: State the constants

$c_1 = 3,\ c_2 = 5,\ n_0 = 64$

Thus $3n^2 \leq f(n) \leq 5n^2$ for every $n \geq 64$. By the formal definition, $f(n) = \Theta(n^2)$.

(f) Prove f(n) is Not O(n)

To show that f(n) is not O(n), suppose for contradiction that f(n) = O(n). By the definition of Big-O, there would be constants $c > 0$ and n_0 such that $f(n) \leq cn$ for all $n \geq n_0$.

Step 1: Use the lower bound

- **$f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$**

Step 2: Combine it with the assumed upper bound

If $f(n) \leq cn$, then $3n^2 \leq cn$. For $n > 0$, divide both sides by n:

- **$3n \leq c$**

Step 3: Show the contradiction

The inequality $3n \leq c$ cannot hold for every sufficiently large n because c is a fixed constant while n grows without bound. Choose any n greater than c/3 and also greater than n_0; then $3n > c$, contradicting the assumed inequality.

Therefore no constant c can upper-bound f(n) by a linear function for all sufficiently large n. Hence f(n) is not O(n).

Conclusion of the proof

The lower and upper bounds establish $\Theta(n^2)$, while the contradiction establishes that the function grows faster than any constant multiple of n. Both results follow directly from the dominant quadratic term.

(g) Scalability Decision — Operation Counts

The platform must process n = 1,000,000 products. The server rate is 10 basic operations per second. The nightly processing window is 10 minutes, which is 600 seconds. At the stated rate, the window can accommodate about 6,000 basic operations.

M1: Duplicate-listing detector

- **$C_1 = n(n - 1)/2$**

  $= 1{,}000{,}000 \times 999{,}999 / 2$

  $= 499{,}999{,}500{,}000$ comparisons

At 10 operations per second, time = 499,999,500,000 / 10 = 49,999,950,000 seconds. This is approximately 1,584 years using 31,536,000 seconds per year. It fails the deadline by an enormous margin.

M2: API rate limiter

For n = 1,000,000, ceil($\log_2 n$) = 20, because 2^{19} = 524,288 and 2^{20} = 1,048,576.

A(1,000,000) = $2^{20} - 1$ = 1,048,575 audits

Time = 1,048,575 / 10 = 104,857.5 seconds, or approximately 29.1 hours. It fails the 10-minute window at the stated processing rate.

S2: Binary search

A binary search requires about 20 comparisons in the worst case for one million entries. At 10 operations per second, 20 operations take about 2 seconds under the simplified model. S2 meets the deadline for an individual lookup.

(g) Scalability Decision — M4 and M5

M4 current design

The recurrence is $\Theta(n^2)$. For an illustrative dominant-term estimate, $n^2 = (1{,}000{,}000)^2 = 1{,}000{,}000{,}000{,}000$ operations. The recurrence with a unit base case has a leading quadratic coefficient, so an exact recurrence estimate may be closer to $2n^2$; either estimate is far beyond the deadline.

Dominant estimate: 10^{12} operations / 10 = 10^{11} seconds

This is approximately 3,170 years using the dominant-term estimate. The current design fails and must be redesigned.

M4 proposed redesign

The recurrence becomes $\Theta(n \log n)$. With $\log_2(1{,}000{,}000) \approx 19.93$, $n \log_2 n$ is approximately 19,931,569 operations.

19,931,569 / 10 ≈ 1,993,157 seconds ≈ 23.1 days

The exact recurrence can add lower-order linear work, making the estimate slightly larger. The redesigned algorithm is much better asymptotically, but at the specified rate it still misses a 10-minute deadline for one million records.

M5: Sales analytics

M5 has $\Theta(n \log^2 n)$ cost. Since $\log_2(1{,}000{,}000) \approx 19.93$, $n \log_2^2 n$ is approximately 397 million operations.

397,000,000 / 10 ≈ 39,700,000 seconds ≈ 1.26 years

This is an order-of-growth estimate; constants and the base-case cost affect the exact value. Nevertheless, it is clearly much longer than 10 minutes.

Deadline capacity

At 10 operations per second, 600 seconds allow only 6,000 operations. M1, M2, both M4 estimates, and M5 exceed that limit. S2 is the only listed workload that meets it for one lookup.

Consolidated Results Table

Module / routine	Operation count or recurrence	Time complexity	Decision at n = 1,000,000
M1 duplicate detector	$n(n-1)/2 = 499{,}999{,}500{,}000$	$\Theta(n^2)$	Fails badly; redesign
M2 rate limiter	$2^{\lceil \log_2 n \rceil} - 1 = 1{,}048{,}575$	$\Theta(n)$	Fails at stated rate; optimise/capacity
M3 S1 sequential search	Best 1; worst n	$\Theta(n)$ worst	Not ideal for large catalogues
M3 S2 binary search	About 20 comparisons	$\Theta(\log n)$	Meets per-lookup target
M4 current	$4T(n/2)+n$	$\Theta(n^2)$	Fails; remove replication
M4 redesigned	$2T(n/2)+n$	$\Theta(n \log n)$	Much better, still misses target at stated rate
M5 sales analytics	$2T(n/2)+n \log n$	$\Theta(n \log^2 n)$	Fails; optimise repeated work
Report generator	$3n^2+10n \log n+50$	$\Theta(n^2)$, not $O(n)$	Quadratic growth is unsuitable at scale

Key mathematical findings

- M1 exact pair comparisons: $n(n-1)/2$.
- M2 exact audit count: $2^{\lceil \log_2 n \rceil} - 1$; tight bound $\Theta(n)$.
- S2 recurrence: $T(n) = 1 + \log_2 n$ for n a power of two; $\Theta(\log n)$.
- M4 current: $\Theta(n^2)$; redesigned: $\Theta(n \log n)$.
- M5: $\Theta(n \log^2 n)$.
- Report generator: $3n^2 \le f(n) \le 5n^2$ for $n \ge 64$; hence $\Theta(n^2)$.

Conclusion

The scalability audit shows that asymptotic analysis is essential when an e-commerce catalogue grows. M1's pairwise comparison routine performs $n(n-1)/2$ checks and becomes impractical at one million products. A hash-based or sorting-based duplicate detector should replace it.

M2 is linear rather than $\Theta(n \log n)$, because its inner-loop work forms a geometric series. However, linear complexity alone does not guarantee that the workload meets the deadline: at the stipulated rate, more than one million audit operations take about 29.1 hours.

Binary search is highly suitable for product lookup when a sorted index is maintained. Its comparison count grows logarithmically, requiring about 20 comparisons for one million products. The Master Theorem shows that removing replication from M4 improves the recommendation engine from $\Theta(n^2)$ to $\Theta(n \log n)$. M5's recurrence yields $\Theta(n \log^2 n)$, and the report generator is $\Theta(n^2)$, not $O(n)$.

The practical recommendation is to prioritise algorithm redesign, then benchmark constants and operation rates. Under the assignment's simplified rate of 10 operations per second, only S2 meets the 10-minute window for an individual lookup. M1, M2, both M4 designs, and M5 require redesign, workload reduction, or substantially greater processing capacity to meet the same deadline.